

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

0

PHASE ZERO

Foundations: What Is an API?

Build a rock-solid mental model of how the web works — clients, servers, HTTP, REST, JSON and where FastAPI fits — before writing a single line of code.

Contents

0.1 What Is the Internet?	5
The restaurant analogy	6
What happens when you type a URL into a browser?	6
0.2 What Is HTTP?	7
HTTP is a language	7
Anatomy of an HTTP Request	7
Anatomy of an HTTP Response	8
HTTP Methods (the verbs)	9
HTTP Status Codes (the response summary)	9
Hands-On 0a: See a real HTTP request with your own eyes	11
0.3 What Is an API?	11
The definition	11
The waiter analogy (this one is worth memorizing)	12
Why do we even need APIs?	12
Web API vs Library API — an important distinction	12
Real-world examples	13
0.4 What Is REST?	13
The words	13
The 6 REST constraints (the brief tour)	14
What makes an API "RESTful"? Resources, nouns, and URLs	14
URL design best practices — a full reference table	15
JSON — the lingua franca of APIs	16
0.5 What Is FastAPI?	16
The Python web framework landscape	16
Why FastAPI?	17
FastAPI vs Flask — side by side	17

What you're going to build	18
Hands-On Exercise 0: Call a real, live API yourself	19
Checkpoint: At This Point You Can Now...	21
Phase 0 Summary	22

Your Journey

0	Foundations: What Is an API?	YOU ARE HERE
1	Setup & Your First API	
2	Path Params, Query Params & Body	
3	Pydantic: Data Validation	
4	Database Integration (SQLAlchemy)	
5	Authentication & Authorization	
6	Advanced FastAPI Features	
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

A quick word before we start. If you've only ever written Python scripts that run on your own machine — printing things, looping over lists, defining classes — then the web can feel like a foreign country with its own language and customs. That feeling is completely normal. Every backend developer alive was once confused by this exact material. The trick is that there are really only **four or five core ideas**, and once they click, everything else is just detail. We'll take them one at a time, slowly, with pictures and analogies. No code yet — just understanding. Let's go.

0.1 What Is the Internet?

Let's start with the simplest possible mental model.

The restaurant analogy

Imagine you walk into a restaurant. You sit down, a **waiter** comes over, you order "a margherita pizza," the waiter walks to the **kitchen**, the kitchen cooks it, and the waiter brings the pizza back to your table.

That entire interaction is the internet in miniature:

- **You (the customer)** = the **client**. You *want* something but you can't make it yourself.
- **The kitchen** = the **server**. It has the ingredients, the ovens, the recipes — the ability to *produce* what you asked for.
- **The waiter** = the **connection** carrying your request there and the result back.

A **client** is any program that *asks* for something: your web browser, a mobile app, a Python script. A **server** is any program that *waits to receive* requests and *responds* to them. The whole internet is millions of clients asking millions of servers for things, all day, every day.

TIP

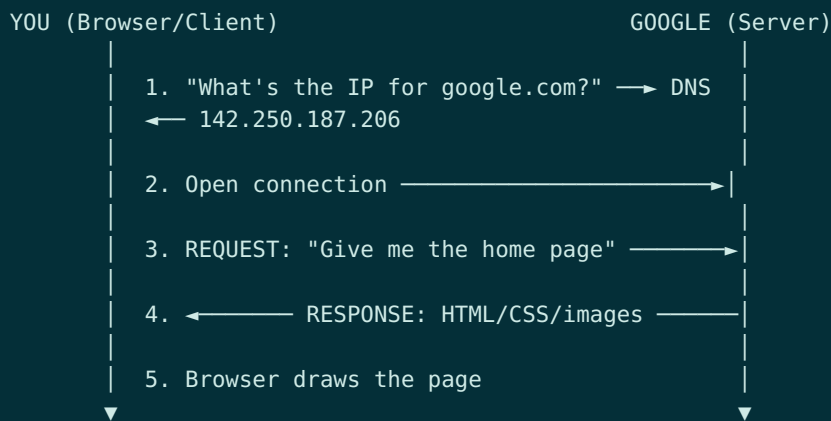
Tip: "Server" is one word doing two jobs in everyday speech. Sometimes it means the *physical computer* sitting in a data center. Sometimes it means the *program* running on that computer (e.g. "the FastAPI server"). When we say server in this tutorial, we almost always mean the **program**. The program is what you're going to write.

What happens when you type a URL into a browser?

When you type `https://www.google.com` and hit Enter, a surprising number of things happen in under a second. Here's the step-by-step, simplified:

1. **You type a URL.** A URL (Uniform Resource Locator) is just an address — like a postal address for a thing on the internet.
2. **DNS lookup.** Computers don't actually understand `google.com`; they understand numeric addresses called IP addresses (like `142.250.187.206`). So your browser first asks a **DNS server** ("the internet's phone book"): "*What's the IP address for google.com?*" and gets a number back.
3. **Connection opened.** Your browser opens a connection to that IP address — like dialing the phone number you just looked up.
4. **Request sent.** Your browser sends a **request**: "*Please give me the home page.*"
5. **Server responds.** Google's server receives the request, figures out what you want, and sends back a **response** — the HTML, CSS, and images that make up the page.
6. **Browser renders.** Your browser takes that response and *draws* it on your screen as the page you see.

DIAGRAM



CONCEPT CHECK

Concept Check: In one sentence, what is the difference between a client and a server?

*Answer: A client **asks** for things; a server **waits for requests and responds** to them. The same computer can be both at different moments.*

That whole "request → response" dance is the heartbeat of the web. Everything from here builds on it. And the rules for *how* that request and response are written down are defined by something called **HTTP**.

0.2 What Is HTTP?

HTTP is a language

If the client and server are going to talk, they need to agree on a **language** — exact rules about how a message is structured, so both sides understand each other. That language is **HTTP** (HyperText Transfer Protocol). A "protocol" is just a fancy word for "an agreed-upon set of rules for communication." (Diplomats follow protocol; so do computers.)

Every interaction in HTTP is a pair: the client sends a **request**, the server sends back a **response**. Always. One request, one response. Let's dissect each.

Anatomy of an HTTP Request

An HTTP request has four parts:

Part	What it is	Example
Method	The <i>verb</i> — what you want to do	<code>GET</code>
URL (path)	The <i>noun</i> — what you want to do it to	<code>/users/42</code>
Headers	Extra info <i>about</i> the request (metadata)	<code>Authorization: Bearer abc123</code>
Body	The actual <i>data</i> you're sending (optional)	<code>{"name": "Ada"}</code>

Here's what a raw request actually looks like on the wire:

```
HTTP

GET /users/42 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGci...
```

Read it like a sentence: "Using the **GET** method, fetch the resource at **/users/42**. By the way (headers), I'd like the response as JSON, and here's my access token to prove who I am." This particular request has no body, because `GET` requests are just asking for data — they don't send any.

Anatomy of an HTTP Response

The server replies with three parts:

Part	What it is	Example
Status Code	A 3-digit number summarizing <i>what happened</i>	<code>200</code>
Headers	Metadata about the response	<code>Content-Type: application/json</code>
Body	The actual data being returned	<code>{"id": 42, "name": "Ada"}</code>

```
HTTP

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 31

{"id": 42, "name": "Ada Lovelace"}
```

Read it: "Everything went fine (**200 OK**). The data I'm sending is JSON (header). Here it is (body)."

TIP

Tip: Notice the symmetry — both requests and responses have **headers** and an optional **body**. The request adds a **method + URL**; the response adds a **status code**. If you remember that shape, you understand 90% of HTTP.

HTTP Methods (the verbs)

The **method** tells the server what *kind* of action you want. There are five you'll use constantly. The analogy: think of a server as a filing cabinet full of records.

Method	Plain English	Filing cabinet analogy	Real-world example
GET	"Give me data"	Read a file (don't change it)	Loading a blog post
POST	"Here's new data, create it"	Add a brand-new file	Submitting a signup form
PUT	"Replace this entire record"	Throw out the old file, put a new one in its place	Overwriting your whole profile
PATCH	"Update just this one field"	Edit one line inside the file	Changing only your email
DELETE	"Remove this"	Shred the file	Deleting a tweet

WARNING

Warning — the most common beginner confusion: **PUT** vs **PATCH**. **PUT** replaces the **whole** resource (you must send every field, even the ones that didn't change). **PATCH** updates **only the fields you send**. If you **PUT** a user object but forget to include their phone number, you may *erase* their phone number. With **PATCH**, fields you don't mention are left untouched.

CONCEPT CHECK

Concept Check: You want to change *only* a product's price and nothing else. Which method?
Answer: **PATCH** — it updates a single field without disturbing the rest.

HTTP Status Codes (the response summary)

The status code is the server's one-number verdict on your request. They come in ranges, and the **first digit tells you the category**:

- **2xx = Success** ("It worked!")
- **4xx = Client error** ("You did something wrong.")

- **5xx = Server error** ("I did something wrong.")

Here are the ones you must know by heart:

Code	Name	Meaning	When you'll see it
200	OK	Success, here's your data	A successful GET
201	Created	Success, and I made a new thing	A successful POST
204	No Content	Success, but I have nothing to send back	A successful DELETE
400	Bad Request	Your request was malformed	Missing required data
401	Unauthorized	You're not logged in / no valid token	Calling a protected route with no token
403	Forbidden	You're logged in, but not allowed	A normal user hitting an admin-only route
404	Not Found	That resource doesn't exist	Asking for <code>/users/999999</code>
422	Unprocessable Entity	Data is the wrong <i>shape</i> or <i>type</i>	Sending <code>"age": "hello"</code> when a number was expected
500	Internal Server Error	The server code crashed	A bug in <i>your</i> code

PHASE 0 • KEY TAKEAWAYS

TIP

Tip — remembering 401 vs 403: 401 = "I don't know who you are" (Unauthorized — really means *unauthenticated*). **403 = "I know exactly who you are, and you're still not allowed"** (Forbidden). The naming is a famous historical quirk; just memorize the distinction.

TIP

Tip: **422** is FastAPI's signature status code. You'll see it *a lot*. It almost always means "the JSON you sent didn't match what the server expected" — wrong type, missing field, value out of range. When you see 422, read the response body: FastAPI tells you *exactly* which field is wrong and why. We'll lean on this heavily in Phase 3.

HANDS-ON

Hands-On 0a: See a real HTTP request with your own eyes

You don't need to install anything for this — you already have a browser, and your browser has a built-in HTTP inspector called **DevTools**.

1. Open your web browser (Chrome, Edge, or Firefox).
2. Open **Developer Tools**: - **Windows/Linux**: press **F12** (or **Ctrl + Shift + I**) - **Mac**: press **Cmd + Option + I**
3. Click the **Network** tab at the top of the panel that appears.
4. With DevTools still open, type **https://www.google.com** in the address bar and press Enter.
5. Watch the Network tab fill up with rows. Each row is one HTTP request your browser made.
6. Click the **very first row** (usually named **google.com** or **www.google.com**).
7. Look at the panel that opens. Find these sections: - **Headers** → **General**: you'll see **Request Method: GET** and **Status Code: 200 OK**. - **Request Headers**: the metadata your browser sent. - **Response Headers**: the metadata Google sent back.

What just happened? You just watched the exact request/response cycle from section 0.1 happen for real. Your browser sent a **GET** request, Google's server replied with **200 OK** and a body full of HTML. Everything in this tutorial is built on what you just saw. Sit with that for a second — the abstract diagram is now a concrete thing you can point at.

CONCEPT CHECK

Concept Check: In the Network tab, you probably saw *dozens* of requests, not just one. Why?
Answer: A modern web page isn't one file. The first request gets the HTML, and that HTML says "I also need this CSS file, these images, this JavaScript" — so the browser fires off a separate request for each. One page = many requests.

0.3 What Is an API?

The definition

API stands for **Application Programming Interface**. That's a mouthful, so let's break the words down:

- **Application** — a program (Twitter, Stripe, your weather app).
- **Programming** — meant to be used *by other programs*, not by humans clicking buttons.

- **Interface** — a defined "front door" through which you interact with something without needing to know what's behind it.

Put simply: **An API is a contract that lets one program request services or data from another program in a predictable way.**

The waiter analogy (this one is worth memorizing)

Back to the restaurant. You (the **diner / client**) want food. The **kitchen (server)** can make it. But you don't walk into the kitchen, grab a pan, and start cooking — that would be chaos. Instead, there's a **waiter**, and there's a **menu**.

- The **waiter** is the **API**. It's the only thing you talk to. You give it requests; it brings back results.
- The **menu** is the **API documentation**. It tells you exactly what you're allowed to ask for and in what format.

You don't need to know how the kitchen works — what brand of oven they use, how the chef organizes the fridge. You just need to know the menu. That's the whole point of an API: it **hides the complicated internals** and gives you a simple, reliable way to ask for things.

DIAGRAM

```
CLIENT —"GET /pizza"—> API —> SERVER (kitchen)
(diner)   ←      pizza — (waiter) ←  cooks the pizza
```

Why do we even need APIs?

Two big reasons:

1. **Separation of concerns.** The team building the iPhone app, the team building the website, and the team building the database don't have to be the same people or even use the same programming language. They all just agree on the API contract and work independently. The waiter doesn't care if the diner speaks English or French, as long as they order from the menu.
2. **Reusability.** Write the API once, and every client can use it — your web app, your mobile app, a partner company's software, a script you wrote at 2am. One backend, many front doors.

Web API vs Library API — an important distinction

The word "API" gets used for two related-but-different things, and beginners often conflate them:

	Library API	Web API
Where it runs	In <i>your</i> program, same machine	On a <i>remote</i> server, over the internet
How you call it	Import it and call a function: <code>math.sqrt(16)</code>	Send an HTTP request over the network
Example	Python's <code>requests</code> library, <code>pandas</code>	The Twitter API, the Stripe API
Speed	Instant (it's local)	Slower (data travels over the network)

When *this tutorial* says "API," we almost always mean **Web API** — something you talk to over HTTP, exactly the kind you're about to build with FastAPI.

Real-world examples

- **Stripe API** — your e-commerce site sends a request like "charge this card £20," and Stripe handles all the terrifying complexity of payments and banking. You never touch a credit card number directly.
- **Google Maps API** — apps send "give me directions from A to B" and get back a route, without building their own maps of the entire planet.
- **Twitter/X API** — programs can post tweets, read timelines, and search, all without a human clicking anything.

Every one of these is a company saying "here's our menu (the API), order what you like."

COMMON MISTAKES

Common Mistakes: - "An API is a website." No — a website is for humans to look at; an API is for programs to talk to. (Though a website is often *powered by* an API behind the scenes — which is exactly the architecture you're learning.) - "I need to understand how Stripe works internally to use its API." No — that's the entire benefit. You only need the menu (the docs), not the kitchen.

0.4 What Is REST?

You'll hear the phrase "REST API" constantly. Let's demystify it.

The words

REST = **RE**presentational **S**tate **T**ransfer. Even spelled out it's cryptic, so here's the plain-English version: REST is a **style** — a set of conventions — for designing web APIs so

they're predictable, consistent, and easy to understand. It's not a law or a piece of software; it's a way of organizing things that the industry mostly agreed on. An API that follows these conventions is called **RESTful**.

The 6 REST constraints (the brief tour)

REST defines six guiding principles. You don't need to memorize these — just understand the *vibe*. The two that matter most for daily work are in **bold**.

1. **Client-Server** — the client and server are separate and independent (we've already covered this).
2. **Stateless** — *each request contains everything the server needs to fulfill it*. The server does **not** remember previous requests. Every request stands alone. (This is huge and we'll revisit it in Phase 5 — it's *why* APIs use tokens instead of login sessions.)
3. **Cacheable** — responses can declare whether they're allowed to be saved and reused, to improve speed.
4. **Uniform Interface** — resources are accessed in a consistent, predictable way (more on this below — it's the heart of REST in practice).
5. **Layered System** — there can be middlemen (proxies, load balancers) between client and server without either side caring.
6. **Code on Demand** (optional) — servers can send executable code to clients (rarely used; safe to ignore for now).

CONCEPT CHECK

Concept Check: "Stateless" means the server doesn't remember you between requests. So how does a server know you're logged in on your second request? *Answer (preview of Phase 5): you send proof of identity — a token — with **every single request**. The server re-verifies it each time. It never "remembers" you from last time.*

What makes an API "RESTful"? Resources, nouns, and URLs

Here is the single most important practical rule of REST:

URLs should name *things* (nouns), and HTTP methods should describe the *action* (verbs). Don't put verbs in your URLs.

A "thing" in REST is called a **resource** — a user, a book, an order. Each resource gets a URL. The *method* says what you're doing to it.

The wrong way (verbs in the URL):

DIAGRAM

```
GET /getUser?id=42
POST /createUser
POST /deleteUser?id=42
GET /listAllBooks
```

The RESTful way (nouns in the URL, verb is the method):

DIAGRAM

```
GET /users/42      ← get user 42
POST /users        ← create a new user
DELETE /users/42   ← delete user 42
GET /books         ← list all books
```

See how clean that is? The same noun `/users` does four jobs depending on the method. You don't invent a new URL for every action — you reuse the resource URL and *change the verb*.

URL design best practices — a full reference table

Goal	Method + URL	Notes
List all books	GET <code>/books</code>	Plural noun for collections
Get one book	GET <code>/books/42</code>	ID identifies the specific one
Create a book	POST <code>/books</code>	POST to the <i>collection</i>
Replace a book	PUT <code>/books/42</code>	Send the whole object
Update part of a book	PATCH <code>/books/42</code>	Send only changed fields
Delete a book	DELETE <code>/books/42</code>	
Get a book's reviews	GET <code>/books/42/reviews</code>	Nesting shows ownership
Filter / search	GET <code>/books?genre=scifi</code>	Use query params, not new URLs
Paginate	GET <code>/books?page=2&size=20</code>	Query params again

TIP

Tip: Use **plural nouns** for collections (`/books` , not `/book`). It reads naturally: `/books/42` = "the books collection, item 42." Be consistent across your whole API — `/books` and `/users` and `/orders` , never a surprise singular `/person` hiding among them.

JSON — the lingua franca of APIs

When the server sends data back, it needs a format both sides understand. The overwhelming winner is **JSON** (JavaScript Object Notation). Don't let the "JavaScript" in the name fool you — every language, including Python, reads and writes JSON effortlessly.

JSON looks *almost exactly* like a Python dictionary:

JSON

```
{
  "id": 42,
  "title": "The Pragmatic Programmer",
  "author": "Hunt & Thomas",
  "year": 1999,
  "in_stock": true,
  "tags": ["programming", "career"],
  "publisher": {
    "name": "Addison-Wesley",
    "country": "USA"
  }
}
```

JSON supports a small set of types: **strings** (`"text"`), **numbers** (`42` , `3.14`), **booleans** (`true` / `false`), **null**, **arrays** (`[...]`), and **objects** (`{...}`). That's it — and it's enough to represent almost anything.

WARNING

Warning — JSON is not quite Python: JSON uses lowercase `true` / `false` / `null` , while Python uses `True` / `False` / `None` . JSON strings *must* use double quotes `"` , never single quotes `'` . FastAPI handles the translation between JSON and Python automatically — but knowing the difference will save you confusion when you read raw responses.

0.5 What Is FastAPI?

You now understand the web. Let's place FastAPI in the landscape.

The Python web framework landscape

A "web framework" is a toolkit that handles all the tedious HTTP plumbing so you can focus on your actual logic. In Python, the three big names are:

Framework	Born	Vibe	Best for
Flask	2010	Tiny, minimal, "bring your own everything"	Small apps, learning, total control
Django	2005	Huge, batteries-included, opinionated	Big full-stack sites with an admin panel, ORM, templates all bundled
FastAPI	2018	Modern, fast, type-driven, API-first	Building APIs (exactly our goal)

Flask is the lightweight classic. Django is the heavyweight that gives you everything (and the kitchen sink). **FastAPI** is the modern newcomer designed specifically for building APIs, and it has taken the Python world by storm.

Why FastAPI?

Four standout reasons, and you'll feel all of them within the first hour of coding:

1. **Automatic interactive docs.** This is the killer feature. FastAPI *reads your code* and generates a beautiful, clickable documentation website for your API — for free, with zero extra work. You can test your endpoints right in the browser. (You'll see this in Phase 1 and it'll feel like magic.)
2. **Type safety via Python type hints.** You write normal Python type hints (`user_id: int`), and FastAPI uses them to **validate incoming data automatically**. Send a string where an int was expected? FastAPI rejects it with a helpful `422` before your code even runs. Fewer bugs, less defensive code.
3. **Speed.** FastAPI is one of the fastest Python frameworks available — comparable to Node.js and Go for many workloads — because it's built on modern async foundations (Starlette and Uvicorn).
4. **Async support built in.** It natively supports `async / await`, so it can handle many simultaneous requests efficiently (great for talking to databases and other APIs without blocking).

TIP

Tip: The "automatic docs" and "automatic validation" both come from the *same source*: your type hints and Pydantic models. Write your data shapes once, and FastAPI gives you validation **and** documentation **and** editor autocomplete from that single definition. This "define once, get everything" philosophy is the soul of FastAPI.

FastAPI vs Flask — side by side

Here's the same "hello world" idea in both, just so you can see the flavor (don't worry about running these yet — that's Phase 1):

PYTHON

```
# — Flask —————
from flask import Flask, jsonify # import the framework
app = Flask(__name__)           # create the app

@app.route("/hello")             # register a route (defaults to GET)
def hello():                     # the handler function
    return jsonify({"msg": "hi"}) # must manually convert to JSON
```

PYTHON

```
# — FastAPI —————
from fastapi import FastAPI      # import the framework
app = FastAPI()                  # create the app

@app.get("/hello")               # register a GET route explicitly
def hello():                     # the handler function
    return {"msg": "hi"}         # plain dict – FastAPI converts to JSON for you
```

Feature	Flask	FastAPI
Auto-generated docs	(add-on needed)	Built in
Data validation	(do it manually)	Automatic via type hints
Async support	Limited	First-class
Type hints used for logic		Core feature
JSON conversion	Manual (<code>jsonify</code>)	Automatic
Learning curve	Gentle	Gentle (and you learn good habits)

Neither is "better" universally — Flask is wonderful and remains widely used. But for building modern, well-documented, validated APIs, FastAPI gives you more for less effort. That's why we're here.

What you're going to build

Throughout this tutorial you'll build the same project over and over, adding a new layer each phase — a **Book Library API** — so each new concept attaches to something you already understand. Then, for the grand finale, you'll build a brand-new, production-grade **TaskFlow API** (a full task-management backend) entirely on your own, using everything you've learned: authentication, a real database, tests, and Docker deployment. By the last page, you'll have a portfolio-worthy project and the skills to build your own.

HANDS-ON

Hands-On Exercise 0: Call a real, live API yourself

Time to talk to a real Web API from the command line. We'll use a free, public, no-signup API called **JSONPlaceholder** that serves fake data for practice.

You'll use **curl** — a command-line tool for making HTTP requests that's already installed on virtually every Mac, Linux, and modern Windows machine.

Step 1 — Open your terminal: - **Mac:** open the *Terminal* app. - **Windows:** open *PowerShell* or *Windows Terminal*. - **Linux:** open your terminal.

Step 2 — Make a GET request:

BASH / TERMINAL

```
# -s = silent (hide the progress bar) so we just see the response body
curl -s https://jsonplaceholder.typicode.com/todos/1
```

Expected output:

JSON

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

Step 3 — See the full request/response, headers and all. Add the **-i** flag to *include* the response headers and status line:

BASH / TERMINAL

```
curl -i -s https://jsonplaceholder.typicode.com/todos/1
```

Expected output (trimmed):

HTTP

```
HTTP/2 200
content-type: application/json; charset=utf-8
...
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

Now label every part — connect it back to what you learned:

- `GET` → the **method** (curl uses GET by default).
- `/todos/1` → the **resource URL**: "the todos collection, item number 1." Classic RESTful design (noun, not verb).
- `HTTP/2 200` → the **status code**: `200 OK`, it worked.
- `content-type: application/json` → a **response header** telling you the body is JSON.
- The `{ ... }` block → the **response body**, the actual data, in JSON.
- Inside it: `"completed": false` → a boolean. `"id": 1` → a number. `"title": "..."` → a string.

Step 4 (optional) — change the resource. Try fetching a different item, and a whole collection:

BASH / TERMINAL

```
curl -s https://jsonplaceholder.typicode.com/todos/5    # one specific todo
curl -s https://jsonplaceholder.typicode.com/users      # the whole users collection
```

Notice that `/todos/5` returns one object, while `/users` returns a JSON **array** `[ {...}, {...}, ... ]` — exactly the "one resource vs collection" distinction from the URL design table.

What just happened? You acted as a **client**. You sent a real **GET request** over **HTTP** to a real **server** across the internet. The server replied with a `200` status and a **JSON body**. You just used a **REST API** by hand — the exact kind of API you'll spend the rest of this tutorial *building*. Everything from 0.1 to 0.4 just happened in two lines on your terminal.

COMMON MISTAKES

Common Mistakes: - *Forgetting the `https://`*: `curl jsonplaceholder...` may fail or behave oddly. Always include the scheme (`https://`). - *Windows quirk*: if `curl` behaves strangely in old `cmd.exe`, use **PowerShell** or **Windows Terminal** instead, where `curl` works as shown. - *Seeing no output*: check your internet connection, and confirm you typed the URL exactly (`/todos/1`, not `/todo/1`).

TIP

Tip — `httpie` (optional, prettier): If you'd like color-coded, easier-to-read output, install `HTTPie`: `pip install httpie`, then run `http GET https://jsonplaceholder.typicode.com/todos/1`. Same idea as curl, friendlier display. Totally optional.

Checkpoint: At This Point You Can Now...

- ☐ Explain the **client/server** model using the restaurant analogy.
- ☐ Describe **step by step** what happens when you type a URL and press Enter.
- ☐ State the four parts of an **HTTP request** (method, URL, headers, body) and the three parts of a **response** (status, headers, body).
- ☐ Choose the correct **HTTP method** for an action (GET/POST/PUT/PATCH/DELETE) and explain PUT vs PATCH.
- ☐ Interpret common **status codes** (200, 201, 204, 400, 401, 403, 404, 422, 500).
- ☐ Define an **API** and explain the waiter analogy and why APIs exist.
- ☐ Explain what **REST** is and design a **RESTful URL** (nouns, not verbs).
- ☐ Read and recognize the structure of a **JSON** payload.
- ☐ Say what **FastAPI** is, why it's compelling, and how it compares to Flask.
- ☐ Make a real API call from your terminal with **curl** and label every part of the response.

If any box isn't checked, scroll back up to that section — it's worth being solid here, because everything from Phase 1 onward assumes this foundation.

Phase 0 Summary

PHASE 0 • KEY TAKEAWAYS

In this phase you built a mental model of the web — no code required.

- The internet is **clients asking servers** for things and getting **responses** back.
- **HTTP** is the language they speak: a **request** (method + URL + headers + body) gets a **response** (status code + headers + body).
- The five core **methods** are GET, POST, PUT, PATCH, DELETE; **status codes** group into 2xx success, 4xx your fault, 5xx server's fault.
- An **API** is a contract — a "waiter" — that lets programs request data/services without seeing the internals.
- **REST** is a design style: name **resources with nouns** in the URL, use the **method as the verb**, and speak **JSON**.
- **FastAPI** is a modern Python framework that turns your **type hints** into automatic **validation** and automatic **interactive docs** — which is exactly why we'll use it.
- You called a live REST API by hand and recognized every part of the response.

You're no longer a stranger to the web. Next phase, you write your first FastAPI app and watch the auto-docs appear.

End of Phase 0. Reply "**continue**" (or with any edits you'd like) and I'll write Phase 1 — Setup & Your First API — in the same full detail.